

WEEK-8

Task-1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
GNU nano 8.7.1                                memory.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a, *b, *temp;

    // malloc
    a = malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    // calloc
    b = calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", b[i]);
    printf("\n");

    // realloc

^G Help          ^O Write Out    ^F Where Is     ^K Cut          ^T Execute
^X Exit          ^R Read File    ^\ Replace      ^U Paste        ^J Justify
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/practical$ nano memory.c
the_boss@DESKTOP-I8C8AJG:~/practical$ gcc -g memory.c -o memory
the_boss@DESKTOP-I8C8AJG:~/practical$ ./memory
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
the_boss@DESKTOP-I8C8AJG:~/practical$ |
```

Report:

The program successfully demonstrated dynamic memory allocation using malloc(), calloc(), realloc(), and free().

Different allocation methods were tested and their behavior was observed.

Valgrind was used to monitor memory usage and identify memory leaks.

The program showed no memory leaks when all allocated memory was properly released.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Code:

```
GNU nano 8.7.1 cow.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int *data = malloc(100000 * sizeof(int));

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 100000; i++)
        data[i] = i;

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Data address: %p\n", (void *)data);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild process:\n");
        printf("Child PID: %d\n", getpid());
        printf("Data address: %p\n", (void *)data);

        printf("Modifying data in child...\n");

        data[0] = 999;
    }
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/practical$ nano cow.c
the_boss@DESKTOP-I8C8AJG:~/practical$ gcc cow.c -o cow
the_boss@DESKTOP-I8C8AJG:~/practical$ ./cow
Before fork:
Parent PID: 1086
Data address: 0x746c3f30a010

Child process:
Child PID: 1087
Data address: 0x746c3f30a010
Modifying data in child...
Child data[0] = 999

Parent process:
Parent data[0] = 0
the_boss@DESKTOP-I8C8AJG:~/practical$ |
```

Report: The Copy-on-Write behavior of fork() was successfully demonstrated. After fork(), the parent and child initially share the same physical memory pages. When the child modified the data, Linux created a private copy of the affected page. The parent retained the original value while the child received the modified value, demonstrating Copy-on-Write.